

passgen

Deterministic Password Generator

Детерминированный генератор паролей

Полная документация проекта для защиты

Подробное описание всех компонентов, функций и возможностей

1. Идея проекта и решаемая проблема

В современном мире каждый человек сталкивается с проблемой паролей. У среднего пользователя 20-30 онлайн-аккаунтов: электронная почта, социальные сети (ВКонтакте, Telegram), рабочие сервисы (GitHub, Jira), образовательные платформы, онлайн-банкинг, интернет-магазины, форумы и многое другое. Для каждого требуется пароль.

Типичные решения этой проблемы имеют серьёзные недостатки:

1. Один пароль на все сервисы — крайне опасно. Если злоумышленник узнает пароль от одного сервиса (например, через утечку базы данных), он получит доступ ко всем аккаунтам. Утечки паролей происходят постоянно: крупные сервисы теряют базы данных с паролями регулярно.
2. Запоминать все пароли — невозможно для большинства людей. Человеческая память не приспособлена хранить десятки случайных строк. Поэтому люди используют простые пароли (123456, password, qwerty) или записывают их на стикерах, что тоже небезопасно.
3. Менеджеры паролей (LastPass, Bitwarden, 1Password) — хранят все пароли в зашифрованной базе. Это удобно, но создаёт единую точку отказа: если злоумышленник получит доступ к мастер-паролю от менеджера, он получит все пароли сразу. Кроме того, менеджеры требуют установки отдельного приложения, синхронизации через облако, и часто являются платными.

passgen предлагает четвёртый подход — детерминированную генерацию. Вместо хранения паролей программа каждый раз вычисляет их заново по криптографическому алгоритму. Пользователь запоминает всего одну фразу (сид), а пароль для любого сервиса восстанавливается по формуле: $\text{пароль} = \text{HMAC-SHA256}(\text{сид}, \text{сервис} + \text{длина} + \text{счётчик})$.

Преимущества этого подхода:

- НЕ НУЖНО ХРАНИТЬ пароли — они всегда восстанавливаются из сида и названия сервиса
- НЕЧЕГО КРАСТЬ — на компьютере нет базы данных с паролями
- НЕЛЬЗЯ ПОТЕРЯТЬ — сид можно восстановить, если записать его в надёжном месте
- РАЗНЫЕ ПАРОЛИ для каждого сервиса — достаточно изменить название в запросе
- НЕ ЗАВИСИТ от интернета — вся генерация происходит локально
- БЕСПЛАТНО и без рекламы — программа с открытым исходным кодом

2. Структура и архитектура проекта

Проект состоит из двух файлов, которые вместе образуют полноценное приложение. Ниже описана структура проекта.

2.1. Состав проекта (файлы)

```
xxxulay/
|-- app.py      # Серверная часть на Python (HTTP-сервер)
|-- index.html  # Клиентская часть (интерфейс и логика)
|-- README.md   # Инструкция по установке и запуску
|-- report.pdf  # Данная документация
|-- .gitignore  # Список игнорируемых Git файлов
```

2.2. Файл app.py — сервер на Python

Назначение: запустить локальный HTTP-сервер и открыть интерфейс в браузере. Файл написан на Python с использованием только стандартной библиотеки (http.server, socketserver, webbrowser, subprocess, shutil, threading, os, sys). Это сделано намеренно, чтобы программа работала на любой системе без установки дополнительных пакетов.

Основные компоненты app.py:

Класс QuietHandler — обработчик HTTP-запросов. Наследуется от SimpleHTTPRequestHandler. Главное отличие: он не выводит логи запросов в консоль (метод log_message переопределён как пустой). Это сделано, чтобы терминал пользователя не засорялся сообщениями вроде "127.0.0.1 - - [08/Jul/2026 12:00:00] GET /index.html". Сервер отдаёт файлы из той же директории, где находится сам скрипт app.py.

Функция open_app_mode(url) — пытается открыть браузер в режиме приложения (--app). Этот режим создаёт отдельное окно без адресной строки, закладок и панели инструментов — выглядит как настоящее настольное приложение. Функция проверяет браузеры в следующем порядке: google-chrome-stable, google-chrome, chromium-browser, chromium, brave-browser, brave, vivaldi, chrome, msedge, microsoft-edge, firefox. Для Chromium-подобных браузеров устанавливается размер окна 420x620 пикселей. Если ни один из этих браузеров не найден, используется функция webbrowser.open() — открывает браузер по умолчанию.

Алгоритм выбора порта: программа пытается запустить сервер на порту 8765. Если этот порт занят другой программой, она последовательно проверяет порты 8766, 8767 и так далее, пока не найдёт свободный. Это гарантирует, что программа всегда запустится, даже если другой экземпляр уже работает.

Многопоточность: сервер запускается в фоновом daemon-потоке, чтобы не блокировать главный поток программы. Главный поток ожидает нажатия Ctrl+C через threading.Event.wait(). При получении сигнала прерывания сервер корректно завершает работу методом httpd.shutdown().

2.3. Файл index.html — клиентская часть

Это основной файл программы, содержащий весь пользовательский интерфейс и всю логику генерации паролей. Он состоит из трёх частей:

HTML (~50 строк): разметка страницы. Все элементы сгруппированы в блоки (карточки): блок сида, блок сервиса, блок параметров (длина, символы, пресеты, расширенные), блок с кнопкой генерации, блок с результатом (логин, пароль, полоска надёжности), блок с кнопкой копирования. В верхней части страницы расположены кнопки экспорта и импорта.

CSS (~90 строк): стилизация. Используются CSS-переменные для цветовой схемы (--bg, --card, --fg, --lilac, --lilac2,

--pink, --crimson, --error и другие). Анимации: fadeUp (появление элементов), glow (пульсация границы при ошибке), slideDown/slideUp (раскрытие панелей), checkPop (анимация переключения кружочков). Дизайн: фиолетово-белая гамма, градиентные кнопки, скруглённые углы (8-12px), тени, скрытый скроллбар.

JavaScript (≈250 строк): вся логика программы. Использует Web Crypto API для криптографических операций. Функции: init(), loadState(), saveCfg(), applyCfg(), toggleInv(), applyPreset(), renderChk(), renderInv(), adj(), toggleSvcPanel(), renderSvcChips(), chip(), gen(), hmacGen(), genLogin(), updateStrength(), cpy(), exportData(), importData(), showImportModal(), confirmImport(), closeImportModal(), toggleAdvanced(). Каждая функция подробно описана в разделе 5.

2.4. Файл .gitignore

```
__pycache__/  
*.рус  
*.руо
```

В .gitignore перечислены файлы и папки, которые не должны попадать в Git-репозиторий: кэш Python (__pycache__) и скомпилированные файлы (*.рус).

3. Криптографический алгоритм (детальное описание)

В этом разделе подробно объясняется, как именно программа вычисляет пароль. Это ключевая часть для понимания работы программы.

3.1. Общая схема

Входные данные:

- seed (сид): секретная фраза, введённая пользователем
- service (сервис): название сайта или приложения
- length (длина): желаемое количество символов в пароле
- charset (алфавит): строка со всеми допустимыми символами

Выходные данные:

- password (пароль): строка длины length, состоящая из символов charset

Дополнительные опции:

- capitalize (первая заглавная): сделать первый символ заглавным
- lastDigit (последняя цифра): заменить последний символ на цифру

3.2. Подробная схема HMAC-генерации

1. Сид преобразуется в массив байтов через TextEncoder.
2. Из этих байтов создаётся криптографический ключ через `crypto.subtle.importKey` с алгоритмом `{name: "HMAC", hash: "SHA-256"}`.
3. Формируется базовая часть сообщения: сервис + длина (как строка).
4. Запускается цикл сбора символов:
 - a. К базовому сообщению добавляется счётчик (`ct = 0, 1, 2...`)
 - b. Вычисляется HMAC-SHA256: `crypto.subtle.sign("HMAC", key, message)`
 - c. Из полученных 32 байтов каждый проверяется:
 - Если байт $< (256 - (256 \% \text{len}(\text{charset})))$, он подходит
 - Подходящий байт: `символ = charset[байт \% \text{len}(\text{charset})]`
 - d. Счётчик увеличивается, пока не наберётся length символов
5. Готовый пароль возвращается.

3.3. Rejection Sampling (отбраковка неподходящих байтов)

Критически важная деталь алгоритма. Дело в том, что 256 (количество возможных значений байта) не делится нацело на длину алфавита (которая может быть, например, 50 или 62). Если просто брать `байт \% \text{len}(\text{charset})` для всех байтов, первые символы алфавита будут встречаться чаще, чем последние. Это создаёт смещение (bias), которое снижает криптостойкость пароля.

Решение: программа вычисляет максимальное значение $\text{max} = 256 - (256 \% \text{len}(\text{charset}))$. Байты, которые $\geq \text{max}$, отбрасываются. Берутся только байты $< \text{max}$. Для них распределение остатка от деления на `len(charset)` будет равномерным. Этот метод называется "rejection sampling" (отбраковочная выборка).

Пример: если в алфавите 50 символов, то $\text{max} = 256 - (256 \% 50) = 256 - 6 = 250$. Байты от 250 до 255 отбрасываются. Байты от 0 до 249 используются. Каждый символ алфавита будет встречаться ровно 5 раз ($250/50 = 5$). Распределение идеально равномерное.

3.4. Инверсия: первая заглавная

Если пользователь включил опцию "Первая заглавная" (`inv.cap = true`), программа берёт первый символ сгенерированного пароля и преобразует его в верхний регистр через JavaScript-метод `.toUpperCase()`. Если первый символ уже заглавный — ничего не меняется.

3.5. Инверсия: последняя цифра

Если пользователь включил опцию "Последняя цифра" (`inv.dig = true`), программа вычисляет дополнительный HMAC только для последней цифры. Это делается отдельно от основного пароля, чтобы не нарушить его детерминированность.

Для вычисления цифры используется:

- Ключ HMAC: сид + `"_lastdig"`
- Сообщение: сервис + длина

Из полученных 32 байтов берётся первый байт, и вычисляется: `цифра = байт % 10`. Последний символ пароля заменяется на полученную цифру.

3.6. Алфавиты наборов символов

Каждый набор символов содержит строго определённый список символов. Важно: из некоторых наборов исключены потенциально путающиеся символы.

Заглавные (capitals):

ABCDEFGHIJKLMNOPQRSTUVWXYZ

(исключены: I, O — путаются с 1 и 0)

Строчные (lowercase):

abcdefghijklmnopqrstuvwxyz

(исключены: j, l, o — путаются с цифрами и друг с другом)

Цифры (digits):

0123456789

Спецсимволы (symbols):

!@#\$%&*+=-.

По умолчанию (при первом запуске) включены: заглавные, строчные, спецсимволы. Цифры выключены. Длина по умолчанию: 10 символов.

3.7. Генерация логина

Логин генерируется отдельно от пароля, через алгоритм SHA-256 (не HMAC).

Входные данные: сид + `"login"` + сервис + счётчик (`loginCounter`).

Счётчик увеличивается при каждом вызове `genLogin()`, поэтому каждый клик даёт новый логин.

Из 32 байтов хеша берутся первые 6 и преобразуются в символы.

Алфавит логина: abcdefghjkmnpqrstuvwxyz23456789

(исключены i, l, o — путаются с 1 и 0; исключены 0, 1 — путаются с буквами).

Длина логина: ровно 6 символов.

4. Интерфейс программы (полный обзор)

В этом разделе описывается каждый элемент пользовательского интерфейса.

4.1. Верхняя панель

В левой части — заголовок "passgen". В правой — две текстовые ссылки: "Экспорт" (скачать настройки в JSON) и "Импорт" (загрузить настройки из JSON). Импорт использует скрытый input type="file", который открывается по клику.

4.2. Блок сида

Заголовок: "Сид". Подпись: "Секретная фраза, которую вы держите в голове." Поле ввода типа text с placeholder "Ваша секретная фраза". Под полем — место для сообщения об ошибке (класс .error, розовый текст). Если поле пустое при попытке генерации — добавляется класс .error, граница поля начинает пульсировать розовым (анимация glow).

4.3. Блок сервиса

Заголовок: "Сервис". Справа от заголовка — кнопка "список" со стрелкой вниз. Поле ввода с placeholder "google, github, vk ...". Под полем — скрытая панель (svc-panel), которая открывается при клике на "список". В панели — сетка svc-grid с чипами сервисов.

4.4. Список сервисов (чипы)

Чипы сервисов бывают двух типов. Популярные сервисы отображаются с цветным кружком-логотипом. Пользовательские сервисы — с серым кружком и вопросительным знаком. Список популярных сервисов:

Google — синий кружок (#4285f4), буква G
 GitHub — чёрный кружок (#333), буквы GH
 VK — синий кружок (#4a76a8), буква V
 Yandex — красный кружок (#fc3f1d), буквы Ya
 Mail — синий кружок (#005ff9), символ @
 iCloud — тёмно-синий кружок (#369), буква i
 Dropbox — синий кружок (#0061ff), буква D
 Telegram — голубой кружок (#0088cc), буква T

При клике на чип: заполняется поле сервиса, закрывается панель списка, применяются сохранённые настройки (если есть). Для сохранённых сервисов (из svcs) работает правый клик для удаления.

4.5. Блок параметров

Верхняя часть: выпадающий список пресетов (select) с четырьмя опциями:

- "Свой" (значение custom) — ручная настройка
- "Минимальный (8, буквы+цифры)" — длина 8, capitals+lowercase+digits
- "Стандартный (10, всё кроме спецсимволов)" — длина 10, capitals+lowercase+digits
- "Максимальный (16, всё)" — длина 16, все 4 набора

Левая колонка: длина пароля (input type="number" от 1 до 99) со стрелками вверх/вниз. Стрелки реализованы как две кнопки (треугольник вверх и вниз), вызывающие adj(1) и adj(-1).

Правая колонка: наборы символов (4 строки с кружочками) и ссылка "Расширенные" со стрелкой, открывающая панель с опциями "Первая заглавная" и "Последняя цифра".

4.6. Блок результата

Поле логина (только для чтения) с кнопкой "Создать логин" справа.

Поле пароля (только для чтения) — моноширинный шрифт, жирное начертание, ширина 100%, выравнивание по центру.

Под полем пароля — индикатор надёжности: три полосы (str-bars) и текстовая рекомендация (str-info).

4.7. Кнопки управления

Основная кнопка "Создать" — градиент lilac->lilac2 (фиолетовый), белый текст, тень. При нажатии: scale(0.97).

Анимация появления: fadeUp с задержкой 0.15s.

Кнопка "Копировать" — градиент violet->#8b5cf6, анимация с задержкой 0.25s.

4.8. Модальное окно импорта

Появляется при импорте JSON-файла. Оверлей (полупрозрачный чёрный фон) с центрированным блоком. В блоке: заголовок "Выберите сервисы для импорта", список сервисов с чекбоксами (все включены по умолчанию), две кнопки: "Отмена" (серая) и "Импортировать" (фиолетовая).

4.9. Цветовая схема (CSS-переменные)

```
--bg:      #f5f3ff (светло-фиолетовый фон)
--card:    #ffffff (белый фон карточек)
--fg:      #1a1a2e (тёмный текст)
--border:  #e5e7eb (светло-серая граница)
--lilac:   #a855f7 (фиолетовый, основной цвет)
--lilac2:  #7c3aed (тёмно-фиолетовый)
--pink:    #f472b6 (розовый — слабый пароль)
--crimson: #e11d48 (малиновый — средний пароль)
--error:   #f43f5e (красный — ошибка)
--shadow:  rgba(90,20,140,.08) (тень карточек)
--hover:   #ede9fe (цвет при наведении)
```

4.10. Анимации

fadeUp: элемент появляется снизу (сдвиг 8px) с прозрачности 0 до 1, длительность 0.4s.

glow: пульсация границы между цветом ошибки (var(--error)) и розовым (var(--pink)), длительность 1.5s, бесконечно.

slideDown/slideUp: раскрытие/сворачивание панели через max-height от 0 до 300px.

checkPop: эффект "пружинки" при переключении кружочка (scale 1 -> 1.2 -> 1).

Все кнопки: при нажатии scale(0.97) для тактильного отклика.

5. Функции JavaScript (полное описание)

В этом разделе подробно описана каждая функция JavaScript, используемая в программе. Описания написаны для человека, который может не знать программирования, но хочет понять, что делает программа и как она работает.

5.1. init() — запуск и подготовка программы

Когда пользователь открывает страницу, программа должна подготовиться к работе. Этим занимается функция `init()`. Она запускается самой последней строкой кода (внизу файла написано просто `init()`).

`init()` делает несколько важных вещей. Сначала она вызывает `loadState()` — очищает все данные от предыдущих запусков. Потом убирает возможные ошибки с поля сида — снимает красную подсветку и очищает текст ошибки.

Затем она привязывает к полям специальные обработчики событий. Поле сида: при вводе текста ошибка исчезает; при нажатии Enter запускается генерация пароля. Поле сервиса: при нажатии Enter запускается генерация; при потере фокуса (`blur`) программа запоминает, какой сервис сейчас выбран.

После этого `init()` проверяет, есть ли сохранённые настройки для текущего сервиса. Если есть — восстанавливает их (длина, чекбоксы, инверсия). Если нет — сбрасывает на стандартные: заглавные + строчные + спецсимволы, длина 10, сервис пустой.

В конце `init()` отрисовывает список сервисов — вызывает `renderSvcChips()`, которая создаёт на экране все чипы с логотипами.

5.2. loadState() — очистка данных

Эта функция нужна для безопасности. Она гарантирует, что при каждом запуске программа начинается с чистого листа. Никакие данные от предыдущего использования не сохраняются.

loadState() удаляет всё, что могло остаться в памяти браузера (localStorage), обнуляет объект настроек perSvc, очищает список сервисов svcs, устанавливает текущий сервис в "(empty)", задаёт стандартные настройки (заглавные+строчные+специальные символы, без инверсии), ставит длину 10 и очищает поле ввода сервиса.

Эта функция — ключевой элемент безопасности. Даже если кто-то откроет программу после вас, он не увидит, для каких сервисов вы генерировали пароли, и какие настройки использовали.

5.3. saveCfg() — сохранение настроек сервиса

Когда пользователь нажимает кнопку "Создать", программа запоминает, какие настройки сейчас активны, и сохраняет их для текущего сервиса. В следующий раз, когда пользователь выберет этот же сервис, настройки автоматически восстановятся.

saveCfg() берёт текущие значения: состояние всех четырёх чекбоксов (какие наборы символов включены), состояние обеих опций инверсии (первая заглавная, последняя цифра), и текущую длину пароля. Всё это упаковывается в объект и сохраняется в perSvc под названием текущего сервиса.

Настройки хранятся только в оперативной памяти. При закрытии или перезагрузке страницы они исчезают. Это сделано для безопасности.

5.4. applyCfg(cfg) — восстановление настроек

Когда пользователь выбирает сервис из списка (кликает на чип), программа должна применить настройки, которые были сохранены для этого сервиса. `applyCfg()` делает это: она принимает объект с настройками и восстанавливает все параметры.

Функция устанавливает длину пароля, восстанавливает состояние всех четырёх чекбоксов, включает или выключает опции инверсии. После этого перерисовывает чекбоксы (`renderChk`) и инверсию (`renderInv`), чтобы пользователь видел актуальные настройки.

5.5. toggleInv(id) — переключение расширенных опций

Расширенные настройки — "Первая заглавная" и "Последняя цифра" — переключаются кликом. toggleInv() обрабатывает этот клик.

Функция принимает id: "cap" для первой заглавной или "dig" для последней цифры. Она меняет значение в объекте window.inv на противоположное (0 становится 1, 1 становится 0). Затем обновляет внешний вид кружочка — добавляет или убирает класс "on", который закрашивает кружок фиолетовым.

Важно: toggleInv() не сохраняет настройки. Сохранение произойдёт только при нажатии кнопки "Создать". Это экономит ресурсы и предотвращает лишние записи.

5.6. applyPreset() — применение пресета

Пресеты — это готовые комбинации настроек для быстрого выбора. Когда пользователь выбирает пресет из выпадающего списка, applyPreset() применяет соответствующую комбинацию.

Если выбран "Минимальный": длина становится 8, включаются заглавные, строчные и цифры. Если "Стандартный": длина 10, те же наборы. Если "Максимальный": длина 16, включаются все четыре набора (заглавные, строчные, цифры, спецсимволы). Если выбран "Свой" — функция ничего не делает, пользователь настраивает вручную.

После применения пресета функция перерисовывает чекбоксы и сохраняет настройки.

5.7. renderChk() — отрисовка чекбоксов

На экране четыре строки с кружочками и названиями наборов. renderChk() создаёт эти строки динамически, через JavaScript.

Функция проходит по массиву СНК, который содержит четыре элемента: заглавные, строчные, цифры, спецсимволы. Для каждого элемента создаётся строка (div) с кружочком (span) и текстом. Кружочку добавляется класс "on", если набор активен.

Каждая строка — кликабельная. При клике значение набора переключается (`window.state[id] = 1 - window.state[id]`), и функция запускается заново для обновления экрана.

5.8. renderInv() — отрисовка опций инверсии

Эта функция обновляет внешний вид кружочков для первой заглавной и последней цифры. Она просто добавляет или убирает CSS-класс "on" для соответствующих элементов (invCap и invDig), в зависимости от значений window.inv.cap и window.inv.dig.

5.9. adj(delta) — регулировка длины

Рядом с полем длины есть две стрелки. `adj()` обрабатывает клики по ним. Она принимает `delta`: +1 для увеличения, -1 для уменьшения.

Функция берёт текущее значение из поля, прибавляет `delta`, и записывает обратно. Проверяет границы: минимальная длина 1, максимальная 99. Если значение выходит за пределы, оно принудительно устанавливается на границу.

5.10. toggleSvcPanel() — открытие списка сервисов

Кнопка "список" справа от поля сервиса открывает и закрывает панель со списком сервисов. `toggleSvcPanel()` просто переключает CSS-класс "open" на панели. Когда класс есть — панель видна (`max-height: 300px`, `opacity: 1`), когда нет — скрыта (`max-height: 0`, `opacity: 0`). Стрелка рядом с кнопкой поворачивается.

5.11. renderSvcChips() — сборка списка сервисов

Функция отрисовывает все чипы сервисов в панели списка. Сначала она проходит по массиву POPULAR (8 предустановленных сервисов) и добавляет те из них, которых ещё нет в сохранённых (svcs). Затем проходит по массиву svcs и добавляет все сохранённые сервисы.

Для каждого сервиса вызывается chip(), которая создаёт готовый DOM-элемент.

5.12. chip() — создание чипа сервиса

chip() принимает три параметра: item (объект сервиса или строка), hasLogo (показывать ли логотип), deletable (можно ли удалять правым кликом).

Если hasLogo и item — объект из POPULAR, функция создаёт цветной кружок: div с размером 20x20px, border-radius 50%, цвет фона из item.c, текст item.l (буква-логотип), белый цвет текста, жирный шрифт 10px.

Если это пользовательский сервис (hasLogo = false, deletable = true), создаётся серый кружок с вопросительным знаком (цвет фона var(--border), цвет текста #999).

При клике на чип: значение поля сервиса устанавливается в item.n (или в сам item, если это строка), панель списка закрывается, применяются настройки. При правом клике на удаляемом чипе: сервис удаляется из svcs, список перерисовывается.

5.13. gen() — генерация пароля (главная функция)

Это центральная функция всей программы. Она запускается при нажатии кнопки "Создать" или при нажатии Enter в полях сида или сервиса.

gen() выполняет следующие шаги по порядку:

1. Читает значение сида из поля и обрезает пробелы. Если сид пустой — добавляет класс ошибки на поле, показывает сообщение "Введите сид" и прекращает выполнение.
2. Читает значение сервиса из поля. Если пусто — использует "(empty)".
3. Читает длину из поля. Если не число — использует 10.
4. Проверяет, включён ли хотя бы один набор символов. Если нет — показывает ошибку.
5. Составляет алфавит: проходит по всем четырём наборам, для каждого включённого добавляет список символов в общую строку.
6. Вызывает hmacGen(seed, svc, len, alpha) — асинхронную функцию, которая вычисляет пароль. Ждёт результат (await).
7. Если включена первая заглавная — делает первый символ заглавным.
8. Если включена последняя цифра — вычисляет цифру через отдельный HMAC и заменяет последний символ.
9. Записывает пароль в поле pwd.
10. Вызывает updateStrength() для обновления полосок надёжности.
11. Пытается скопировать пароль в буфер обмена. В случае успеха показывает сообщение "Скопировано!" на 2 секунды.
12. Если сервиса нет в списке svcs и он не "(empty)" — добавляет его в список, сортирует и перерисовывает.
13. Устанавливает curSvc = svc.
14. Вызывает saveCfg() для сохранения настроек.

5.14. hmacGen() — криптографическое ядро

Это функция, которая выполняет actual криптографические вычисления. Она асинхронная (async), потому что использует Web Crypto API, который работает асинхронно.

Параметры: seed (сид), svc (сервис), len (длина), alpha (алфавит).

Алгоритм работы:

1. seed преобразуется в байты через `TextEncoder().encode()`.
2. Из байтов импортируется ключ: `crypto.subtle.importKey("raw", keyBytes, {name: "HMAC", hash: "SHA-256"}, false, ["sign"])`.
3. Базовая часть сообщения: `TextEncoder().encode(svc + len)`.
4. Вычисляется максимальное значение $\max = 256 - (256 \% \alpha.length)$ для rejection sampling.
5. Запускается цикл `while (pwd.length < len)`:
 - a. Создаётся сообщение длиной `msgBase.length + 1`, копируется `msgBase`, в последний байт записывается `st` (счётчик, начинается с 0).
 - b. Вычисляется HMAC: `crypto.subtle.sign("HMAC", key, message)`.
 - c. Из результата (`ArrayBuffer`) создаётся `Uint8Array`.
 - d. Для каждого байта: если `байт < max`, то `pwd += alpha[байт \% alpha.length]`.
 - e. Если длина пароля достигла `len` — цикл прерывается.
 - f. `st` увеличивается на 1.
6. Готовый пароль возвращается.

Благодаря асинхронности, браузер не "зависает" во время вычислений. Даже при генерации длинных паролей (99 символов) интерфейс остаётся отзывчивым.

5.15. genLogin() — генерация логина

Создаёт короткий (6 символов) логин, который легко запомнить и продиктовать.

Алгоритм:

1. Проверяет, что сид введён (иначе — ошибка).
2. Проверяет, что сервис указан (иначе — выход).
3. Увеличивает счётчик loginCounter на 1.
4. Формирует строку: seed + "login" + svc + loginCounter.
5. Преобразует в байты и вычисляет SHA-256: crypto.subtle.digest("SHA-256", bytes).
6. Из 32 байтов берёт первые 6.
7. Для каждого байта: login += alpha[байт % alpha.length].
8. Алфавит логина: "abcdefghijklmnopqrstuvwxyz23456789" (32 символа).
9. Записывает результат в поле login.

Каждый клик по кнопке "Создать логин" даёт новый логин, потому что loginCounter увеличивается при каждом вызове. Но при одинаковом сиде+сервисе+счётчике логин будет одинаковым (детерминированность).

5.16. updateStrength() — индикатор надёжности

После генерации пароля эта функция оценивает его качество и показывает результат в виде цветных полосок и текста.

Логика оценки:

1. Получает длину пароля (len) и количество выбранных наборов (sets).
2. Если len $\geq$ 14 и sets $\geq$ 4: уровень "strong" (отличный), эмодзи "uwu", текст "Отличный пароль".
3. Если len $\geq$ 10 и sets $\geq$ 3: уровень "medium" (средний), эмодзи ":3", текст "Хороший, можно длиннее".
4. Иначе: уровень "weak" (слабый), эмодзи ">.<", текст "Добавьте символов или увеличьте длину".
5. Отрисовывает от 1 до 3 полосок в зависимости от уровня. Все полоски одного уровня окрашиваются в одинаковый цвет: слабый = розовый (#f472b6), средний = малиновый (#e11d48), отличный = фиолетовый (#a855f7).
6. Текст рекомендации окрашивается в тот же цвет.

5.17. сру() — копирование в буфер

Копирует текст пароля в буфер обмена, чтобы пользователь мог вставить его на сайте. Использует современный API: `navigator.clipboard.writeText()`. Если этот метод недоступен (старый браузер, не HTTPS), использует запасной: выделяет текст в поле и вызывает `document.execCommand("copy")`.

После копирования показывает сообщение "Скопировано!" на 1.5 секунды.

5.18. exportData() — экспорт в JSON

Собирает все настройки и скачивает их в виде JSON-файла.

Алгоритм:

1. Создаёт пустой объект clean.
2. Проходит по всем сервисам из svcs и по "(empty)", для каждого, если есть настройки в perSvc, добавляет в clean.
3. Формирует объект data = { svcs: svcs, perSvc: clean }.
4. Преобразует в JSON с форматированием: JSON.stringify(data, null, 2).
5. Создаёт Blob с типом application/json.
6. Создаёт виртуальную ссылку (<a>) с href = URL.createObjectURL(blob) и download = "passgen_export.json".
7. Программно кликает по ссылке — начинается скачивание.
8. Удаляет созданную ссылку.

5.19. `importData(event)` — импорт из JSON

Когда пользователь выбирает JSON-файл через кнопку "Импорт", эта функция читает файл и готовит данные к импорту.

Алгоритм:

1. Получает выбранный файл из `event.target.files[0]`.
2. Создаёт `FileReader`.
3. В обработчике `onload`: парсит JSON, проверяет наличие полей `perSvc` или `svcs`. Если данные некорректны — показывает сообщение об ошибке.
4. Если данные корректны — сохраняет их в `importDataCache` и вызывает `showImportModal(d)`.
5. Очищает значение `input file`, чтобы можно было повторно выбрать тот же файл.

5.20. showImportModal(d) — окно выбора сервисов

Создаёт модальное окно со списком всех сервисов из импортируемого файла.

Алгоритм:

1. Берёт список сервисов: `Object.keys(d.perSvc || {}).sort()`.
2. Если список пуст — показывает сообщение "Нет сервисов для импорта" и выходит.
3. Для каждого сервиса создаёт строку (label) с чекбоксом (`input type="checkbox", checked=true`) и названием сервиса.
4. Добавляет все строки в `modallist`.
5. Показывает модальное окно: `document.getElementById("importModal").classList.add("open")`.

5.21. confirmImport() — подтверждение импорта

Когда пользователь нажал "Импортировать" в модальном окне, эта функция загружает выбранные настройки в программу.

Алгоритм:

1. Берёт importDataCache (данные из файла).
2. Собирает все отмеченные чекбоксы из modalList.
3. Если ни один не отмечен — показывает сообщение "Выберите хотя бы один сервис".
4. Для каждого отмеченного: загружает настройки в perSvc[svc], добавляет сервис в svcs (если его там нет).
5. Сортирует svcs, перерисовывает чипы.
6. Если настройки загружены для текущего сервиса — применяет их.
7. Сохраняет состояние (saveState), закрывает модальное окно, показывает сообщение "Импорт завершён!" на 3 секунды.

5.22. closeImportModal() — закрытие окна импорта

Просто убирает модальное окно: `document.getElementById("importModal").classList.remove("open")` и очищает `importDataCache = null`.

5.23. toggleAdvanced() — расширенная панель

Открывает или закрывает панель с опциями инверсии. Работает через CSS: изменяет max-height панели с 0 на 120px (и обратно) и opacity с 0 на 1. Стрелка рядом с текстом поворачивается на 90 градусов при открытии.

5.24. getCfg() — получение настроек сервиса

Вспомогательная функция. Если для сервиса нет сохранённых настроек, создаёт стандартные (заглавные+строчные+специальные символы, длина 10, без инверсии). Возвращает настройки.

5.25. onSvcChange() — обработчик смены сервиса

Вызывается при ручной смене сервиса (например, при вводе текста). Обновляет curSvc и применяет настройки, если они есть.

5.26. saveState() — заглушка безопасности

В текущей версии программы функция `saveState()` пуста. Раньше она сохраняла данные в `localStorage`, но от этого отказались в пользу сессионного режима. Функция оставлена для обратной совместимости, на случай если в будущем понадобится куда-то сохранять данные.

6. Обработка ошибок и граничные случаи

Программа обрабатывает следующие ошибочные ситуации:

1. Пустой сид: при попытке генерации пароля без сида поле подсвечивается розовой пульсирующей границей (анимация glow) и показывается сообщение "Введите сид". Генерация не происходит. При начале ввода ошибка снимается.
2. Пустой набор символов: если пользователь отключил все четыре чекбокса, показывается сообщение "Выберите хотя бы один тип символов". Генерация не происходит.
3. Длина меньше 1 или больше 99: `adj()` принудительно устанавливает значение на границу (1 или 99).
4. Занятый порт: если стандартный порт 8765 занят, программа пробует следующие порты (8766, 8767...) пока не найдёт свободный.
5. Ошибка импорта: если выбранный файл не является корректным JSON или не содержит данных passgen, показывается соответствующее сообщение.
6. Пустой файл импорта: если в импортируемом файле нет ни одного сервиса, показывается сообщение "Нет сервисов для импорта".
7. Ошибка копирования: если Clipboard API недоступен, используется запасной метод `execCommand("copy")`.
8. Сервис не выбран: если при генерации логина не указан сервис, функция `genLogin()` просто завершается без ошибки.

7. Экспорт и импорт данных

7.1. Формат JSON

Файл экспорта/импорта имеет следующую структуру:

```
{
  "svcs": [
    "google",
    "github",
    "vk"
  ],
  "perSvc": {
    "google": {
      "state": {
        "capitals": 1,
        "lowercase": 1,
        "digits": 0,
        "symbols": 1
      },
      "inv": {
        "cap": 0,
        "dig": 1
      },
      "len": 16
    },
    ...
  }
}
```

Поле `svcs` — массив строк с названиями сервисов, для которых есть настройки.

Поле `perSvc` — объект, где каждый ключ — название сервиса, а значение — объект с полями:

`state` — объект с флагами `capitals`, `lowercase`, `digits`, `symbols` (1 = включено)

`inv` — объект с флагами `cap` (первая заглавная), `dig` (последняя цифра)

`len` — длина пароля (число от 1 до 99)

7.2. Поведение при экспорте

Экспортируются только те сервисы, для которых есть сохранённые настройки в `perSvc`. Гарантируется, что в файле не будет дубликатов и пустых записей. Файл форматируется с отступами в 2 пробела для удобства чтения.

7.3. Поведение при импорте

При импорте пользователь видит список всех сервисов из файла и может выбрать только нужные. Выбранные сервисы добавляются в `svcs` (если их там ещё нет) и их настройки копируются в `perSvc`. Если сервис уже существует, его настройки перезаписываются.

8. Безопасность (подробно)

passgen разработан с учётом следующих принципов безопасности:

8.1. Никакие данные не передаются по сети

Веб-сервер работает на localhost (127.0.0.1). Доступ к странице имеет только текущий компьютер. Это гарантирует, что сид, пароли и настройки не могут быть перехвачены при передаче.

8.2. Сессионный режим

Все данные (список сервисов, настройки) хранятся только в оперативной памяти. При закрытии вкладки или перезагрузке страницы все данные безвозвратно удаляются. Это защищает от доступа к истории генерации при повторном открытии.

8.3. Стандартная криптография

HMAC-SHA256 — это проверенный международный стандарт (RFC 2104). SHA-256 является частью семейства SHA-2, разработанного АНБ США и опубликованного NIST. Алгоритм используется в протоколах TLS/SSL, SSH, IPsec, в блокчейне биткоина, в системах электронной подписи.

8.4. Rejection Sampling

Как описано в разделе 3.3, программа использует rejection sampling для обеспечения равномерного распределения символов. Это предотвращает bias, который мог бы снизить энтропию пароля.

8.5. Разделение ключа и сообщения

В HMAC ключ (сид) и сообщение (сервис+длина+счётчик) обрабатываются отдельно. Даже если злоумышленник узнает пароль, он не сможет восстановить сид из-за необратимости хеш-функции.

8.6. Разные пароли для разных сервисов

Поскольку название сервиса является частью входных данных HMAC, изменение одного символа в названии даёт полностью другой пароль. Это гарантирует, что утечка пароля от одного сервиса не скомпрометирует другие.

8.7. Чего программа НЕ делает (ограничения безопасности)

Программа не шифрует данные (она их вычисляет, а не хранит).

Программа не использует соль (salt) — в этом нет необходимости, так как сервис выполняет роль соли.

Программа не защищена от кейлоггеров (как и любой генератор паролей).

Программа не требует мастер-пароля (сид — это и есть мастер-пароль).

Программа не создаёт резервных копий (сид должен храниться отдельно).

9. Установка и системные требования

9.1. Системные требования

Операционная система: Windows, Linux, macOS (любая с поддержкой Python 3).

Python: версия 3.6 или выше.

Браузер: любой современный (Google Chrome, Mozilla Firefox, Microsoft Edge, Brave, Opera, Safari) с поддержкой Web Crypto API.

Дополнительные пакеты: не требуются.

Интернет: не требуется (вся работа локальная).

9.2. Установка на Linux

1. Установить Git (если не установлен): `sudo pacman -S git` (Arch) или `sudo apt install git` (Ubuntu/Debian).

2. Склонировать репозиторий:

```
git clone git@github.com:inzexg-coder/xxxulay.git
cd xxxulay
```

3. Запустить:

```
python3 app.py
```

9.3. Установка на Windows

Вариант 1 (с Git):

1. Установить Git с git-scm.com если не установлен.

2. Открыть командную строку (cmd) или PowerShell.

3. `git clone git@github.com:inzexg-coder/xxxulay.git`

4. `cd xxxulay`

5. `python app.py`

Вариант 2 (без Git):

1. Открыть github.com/inzexg-coder/xxxulay в браузере.

2. Нажать зелёную кнопку "Code" -> "Download ZIP".

3. Распаковать ZIP-архив в любую папку.

4. Открыть эту папку, запустить `python app.py`

9.4. Запуск без открытия браузера

Если программа не открыла окно автоматически, откройте браузер вручную и перейдите по адресу <http://localhost:8765> (или другому порту, указанному в терминале).

9.5. Завершение работы

Нажмите Ctrl+C в терминале, где запущена программа. Сервер корректно завершит работу. Если программа не отвечает, можно закрыть терминал — сервер остановится автоматически.

9.6. Возможные проблемы

Проблема: "Ошибка: Address already in use"

Решение: Программа автоматически выберет другой порт. Если ошибка повторяется, завершите процессы, использующие порты 8765-8770.

Проблема: Не открывается окно браузера

Решение: Откройте браузер вручную и перейдите по адресу из терминала.

Проблема: Окно открывается с адресной строкой

Решение: Установите Chromium-браузер (Chrome, Brave, Edge) для режима `--app`.

Проблема: "python не является внутренней или внешней командой" (Windows)

Решение: Установите Python с python.org, при установке отметьте "Add Python to PATH".